

# CampusGrade

*Java Academic Performance Manager*

**Course:** Programming in Java

**Student:** Vibhor Gupta

**Reg. No.:** 24BCY10335

**Submission date:** 18 September 2026

*A terminal-based Java application for managing students, courses, weighted assessments, academic reports, and file-based persistence.*

## 1. Introduction

CampusGrade is a command-line academic performance manager I wrote in Java. The goal was to bring together the core concepts from the course into one working application rather than a collection of disconnected examples. With it, you can keep student and course records, log weighted assessments, work out course percentages, convert those into letter grades, compute a credit-weighted GPA, and pull up a course summary — all from a terminal.

I kept the project dependency-free on purpose. Data lives in plain text files on disk, so there's no database to set up and nothing to install beyond a standard JDK. Internally, the code is split across packages covering the command-line layer, business rules, analytics, persistence, data models, validation, and exception handling — each with its own clear job.

The VITYarhi instructions ask for a project tied to the subject syllabus that shows implementation, documentation, testing, and version-control practice. I structured CampusGrade as a self-contained Java repository with source packages, tests, sample data, design diagrams, a README, and this report to meet those expectations.

## 2. Problem Statement

Academic marks have a tendency to end up scattered — across spreadsheets, group chats, handwritten notes, or wherever they happened to land. That makes it annoying to answer straightforward questions: which assessments belong to a particular student, what does the weighted course percentage actually work out to, which letter grade does that translate to, or where does the student's GPA sit right now?

**Problem:** Provide a simple terminal-based system that stores academic records, applies weighted assessment rules, validates input, and produces useful performance summaries.

**Proposed solution:** Use Java classes and collections for the in-memory model, a service layer for academic calculations, file I/O for persistence, and a CLI controller for interaction.

## 3. Objectives

- Build an executable Java application that runs directly from a terminal.
- Show object-oriented design through separate model, service, repository, utility, and exception classes.
- Support student management, course management, assessment entry, reporting, analytics, and persistence.
- Apply validation and exception handling so invalid data never silently enters the system.
- Provide test coverage for the main calculation and validation rules.
- Document the project with the diagrams and repository files required for evaluation.

## 4. Functional Requirements

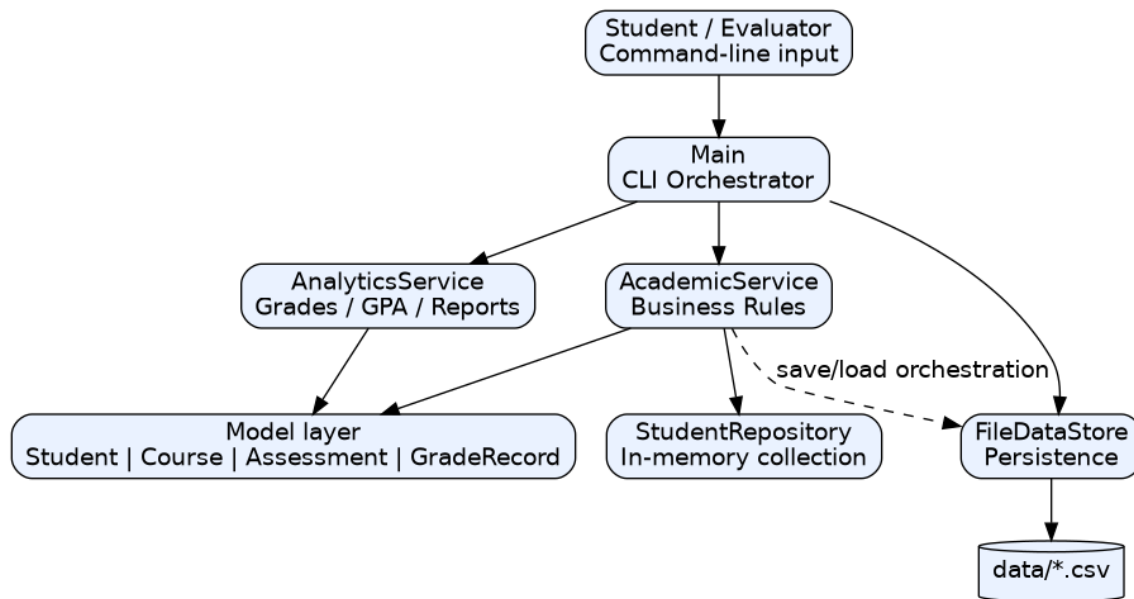
| Module                  | Inputs                                                        | Processing                                                                       | Outputs                                                                 |
|-------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Student management      | ID, name, email, program                                      | Validate fields; reject duplicate IDs; store student                             | Student added / list of students / validation error                     |
| Course management       | Code, title, credits                                          | Validate credits; reject duplicate codes                                         | Course added / list of courses / validation error                       |
| Assessment management   | Student ID, course code, assessment, score, max score, weight | Validate references and numeric rules; store assessment                          | Success message or validation error                                     |
| Analytics and reporting | Student ID or course code                                     | Calculate weighted percentages, grades, GPA, class averages                      | Grade card, GPA, course summary                                         |
| What-if analysis        | Student ID, course code, target percentage                    | Use recorded assessment weight to calculate required average on remaining weight | Current percentage, required remaining percentage, reachability message |
| Persistence             | Save or reload command                                        | Read/write students, courses, assessments                                        | CSV-like files in data/ directory                                       |

## 5. Non-Functional Requirements

| Requirement         | How it's handled                                                                               |
|---------------------|------------------------------------------------------------------------------------------------|
| Usability           | Menu-driven CLI with short commands, predictable prompts, and readable report output.          |
| Reliability         | Validation blocks invalid scores, bad weights, missing references, and duplicate IDs or codes. |
| Performance         | All operations run in-memory; the expected dataset is small enough that actions are immediate. |
| Maintainability     | Model, service, repository, utility, and exception packages each own a distinct concern.       |
| Error handling      | Checked validation exceptions get converted into plain user-facing messages by the CLI layer.  |
| Resource efficiency | No external service or database process required — persistence is just three small text files. |
| Portability         | Only a standard JDK is needed; no OS-specific GUI libraries involved.                          |

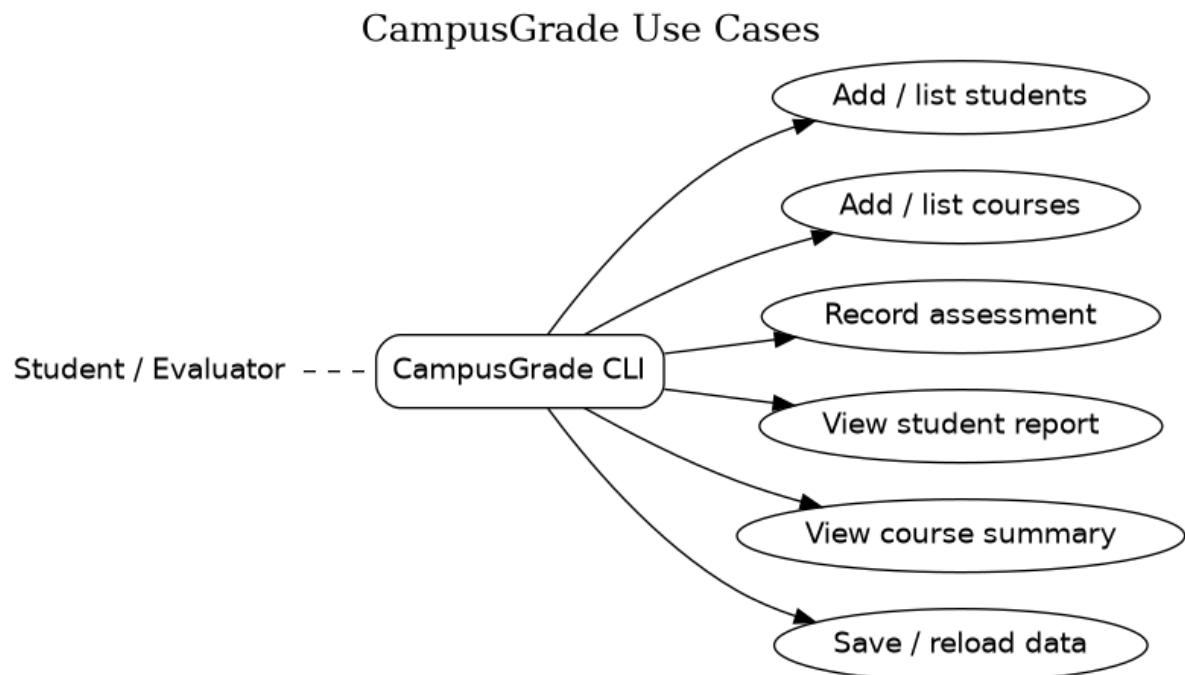
## 6. System Architecture

The application follows a small layered architecture. Main handles terminal interaction. AcademicService owns the business rules and ties together students, courses, and assessments. AnalyticsService handles grade cards, GPA, and class averages. StudentRepository manages the in-memory student collection, while FileDataStore keeps file I/O isolated from everything else. The model package holds the data objects and the GradeRecord result type.



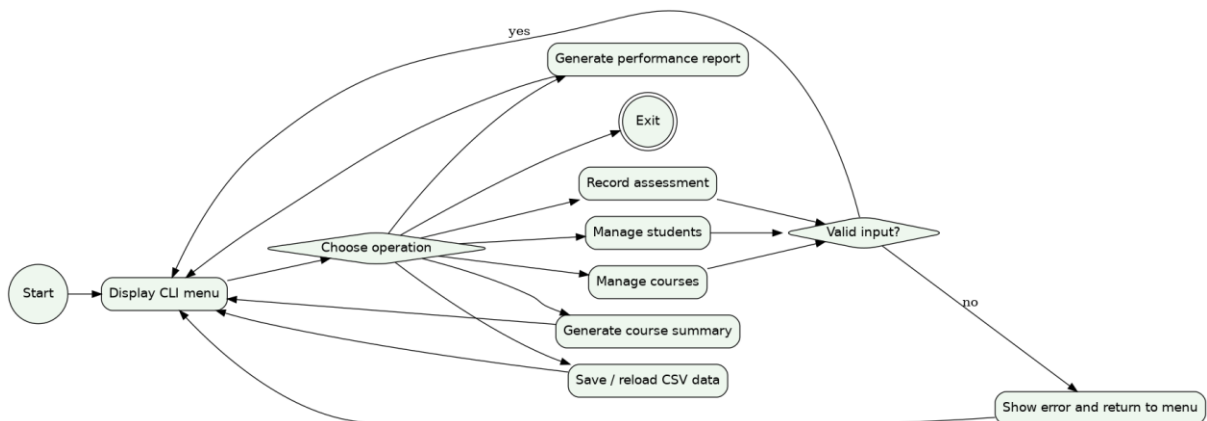
## 7. Design Diagrams

### 7.1 Use Case Diagram



The student or evaluator can trigger all main operations directly from the command-line interface.

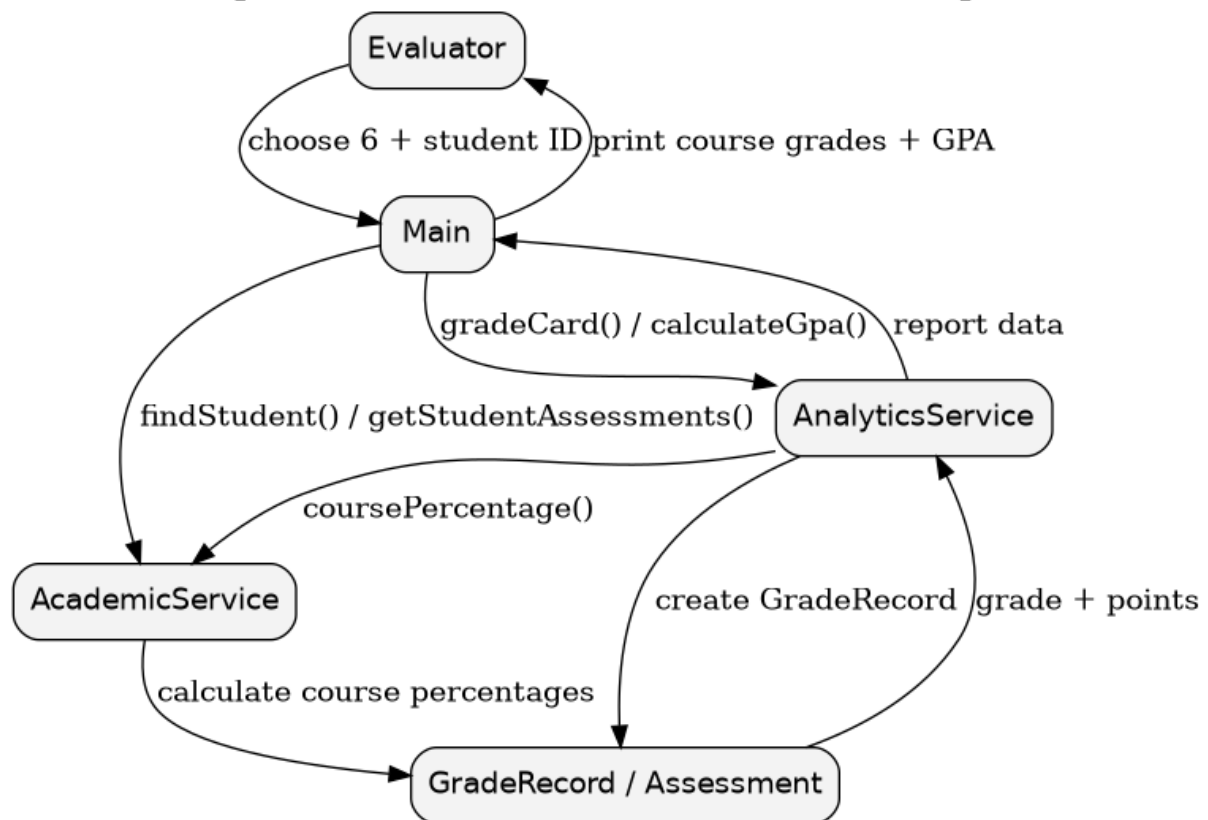
### 7.2 Workflow Diagram



Every data-entry path passes through validation before the application returns to the main menu.

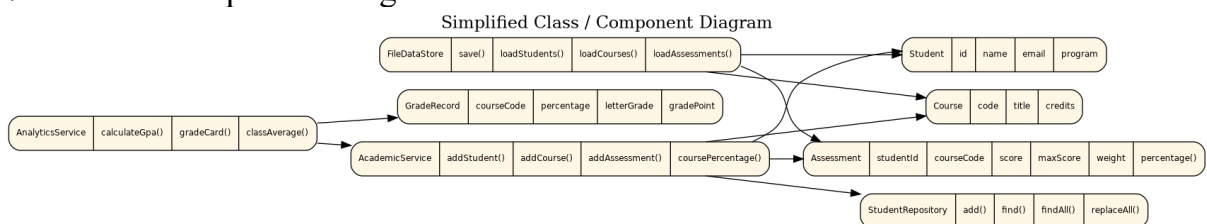
### 7.3 Sequence Diagram — Student Performance Report

#### Sequence: View Student Performance Report



The sequence traces how the CLI requests academic data, the service layer computes percentages, and the analytics layer produces the grade and GPA values shown to the user.

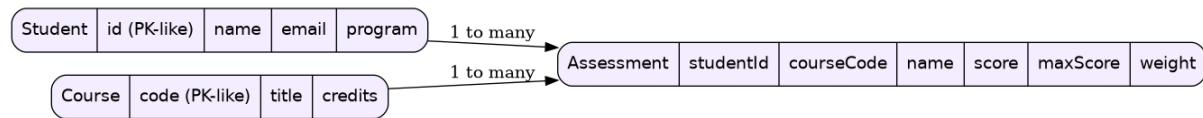
### 7.4 Class / Component Diagram



Classes are organised by responsibility rather than piling everything into one large Main class.

## 7.5 Data / Storage Model

The project uses flat files instead of a relational database. The diagram below is a simple relationship model showing how a student and a course are referenced by assessment records — not an SQL schema.



The flat-file format is deliberately simple: students.csv, courses.csv, and assessments.csv are separate files joined logically by student ID and course code.

## 8. Design Decisions and Rationale

| Decision                   | Why                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------|
| CLI instead of GUI         | Matches the requirement for terminal execution and avoids GUI configuration overhead during evaluation. |
| Standard Java library only | Makes the setup reproducible with any normal JDK and eliminates dependency-download failures.           |
| Layered packages           | Keeps user interaction, business rules, data access, and models separate — easier to read and test.     |
| File-based persistence     | Real persistence without needing a database install or credentials.                                     |
| Weighted assessment model  | Reflects a common academic calculation pattern and makes good use of Java collections and arithmetic.   |
| Dedicated GradeRecord      | Keeps grade/point conversion in one place rather than scattering thresholds across the app.             |
| Custom checked exception   | Makes invalid user input an explicit part of the control flow rather than a runtime surprise.           |

## 9. Implementation Details

### 9.1 Java Source Modules

| File / class             | Responsibility                                                    |
|--------------------------|-------------------------------------------------------------------|
| Main.java                | Menu, prompts, command dispatch, and user-facing errors.          |
| Student.java             | Student domain object and persistence conversion.                 |
| Course.java              | Course domain object and persistence conversion.                  |
| Assessment.java          | Assessment validation, percentage calculation, and serialization. |
| GradeRecord.java         | Letter grade and grade-point rules.                               |
| StudentRepository.java   | In-memory student collection and lookup.                          |
| FileDataStore.java       | Read/write operations for local data files.                       |
| AcademicService.java     | Core academic rules and weighted course calculations.             |
| AnalyticsService.java    | GPA, grade card, and class-average analytics.                     |
| PredictionService.java   | What-if target analysis for the remaining assessment weight.      |
| InputValidator.java      | Reusable command-line field validation and parsing.               |
| ValidationException.java | Custom checked exception for invalid data.                        |
| TestRunner.java          | Assertion-based tests for core behaviour.                         |

### 9.2 Grading Algorithm

- Assessment percentage:  $\text{percentage} = (\text{score} / \text{maxScore}) \times 100$
- Weighted course percentage:  $\Sigma(\text{assessment percentage} \times \text{weight}) / \Sigma(\text{weight})$

- Letter grade mapping:  $A+ \geq 90$  | A: 80–89.99 | B: 70–79.99 | C: 60–69.99 | D: 50–59.99 | F < 50
- GPA: grade points are weighted by course credits across all courses with recorded assessments.
- What-if analysis: if total recorded weight is under 100%, the program estimates the average needed on the remaining weight to hit a chosen target — and flags when the target is already locked in or mathematically out of reach.

### 9.3 Persistence Format

Records are written line by line using | as a field separator. Validation rejects that character inside entered values, which keeps parsing straightforward without pulling in a third-party CSV library.

## 10. Screenshots / Results

```

C:\WINDOWS\system32\cmd. x + v
Weight (%): 20
Assessment recorded successfully.

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 6
Student ID: S1001

Student: Aarav Mehta (S1001)
Program: B.Tech CSE
-----
Course   Percentage   Grade   Grade Point
JAVA101   87.20%      A        9.0
DSA201    79.00%      B        8.0
CSE3003   85.00%      A        9.0
GPA: 8.67 / 10.00

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 0
Goodbye.

D:\VIBHOR\UIT\JAVA\CampusGrade_VITyarthi_Submission\vityarthi_java_project>

C:\WINDOWS\system32\cmd. x + v
Assessment name: Midterm Exam
Score: 85
Maximum score: 100
Weight (%): 20
Assessment recorded successfully.

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 6
Student ID: S1001

Student: Aarav Mehta (S1001)
Program: B.Tech CSE
-----
Course   Percentage   Grade   Grade Point
JAVA101   87.20%      A        9.0
DSA201    79.00%      B        8.0
CSE3003   85.00%      A        9.0
GPA: 8.67 / 10.00

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: |

```

```
C:\WINDOWS\system32\cmd. x + v

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 5
Student ID: S1001
Course code: CSE3003
Assessment name: Midterm Exam
Score: 85
Maximum score: 100
Weight (%): 20
Assessment recorded successfully.

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 6
Student ID: S1001

Student: Aarav Mehta (S1001)
Program: B.Tech CSE
-----
Course   Percentage   Grade   Grade Point
JAVA101   87.20%       A        9.0
DSA201    79.00%       B        8.0

C:\WINDOWS\system32\cmd. x + v

=== CampusGrade - Java Academic Performance Manager ===

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 2

Students
ID      Name           Email           Program
S1001   Aarav Mehta    aarav@example.com B.Tech CSE
S1002   Diya Sharma    diya@example.com  B.Tech CSE
S1003   Rohan Singh    rohan@example.com  B.Tech CSE

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: 3
Course code: CSE3003
Course title: Operating Systems
Credits: 4
Course added successfully.

1. Add student
2. List students
3. Add course

C:\WINDOWS\system32\cmd. x + v

D:\VIBHOR\VIT\JAVA\CampusGrade_VITyarthi_Submission\vityarthi_java_project>java -cp out com.vibhor.campusgrade.Main
Data loaded from data/*.csv

=== CampusGrade - Java Academic Performance Manager ===

1. Add student
2. List students
3. Add course
4. List courses
5. Record assessment
6. Student performance report
7. Course summary
8. What-if target analysis
9. Save data
10. Reload data
0. Exit
Choose: |
```

## 10.1 Example Result

| Operation              | Observed result                                                                   |
|------------------------|-----------------------------------------------------------------------------------|
| List students          | Three sample students load from the data directory.                               |
| Student S1001 report   | JAVA101: 87.20% (A), DSA201: 79.00% (B), GPA: 8.50.                               |
| DSA201 what-if target  | For an 80% target, the sample requires 84.00% on the remaining assessment weight. |
| JAVA101 course summary | Class average 78.20%, with the three sample student percentages listed.           |
| Test suite             | ALL TESTS PASSED.                                                                 |

## 11. Testing Approach

I used a lightweight assertion-based test runner so an evaluator can run the tests without downloading JUnit or anything else — appropriate for a small dependency-free educational project.

| Test                | Purpose                                               | Expected behaviour                                 |
|---------------------|-------------------------------------------------------|----------------------------------------------------|
| Weighted percentage | Checks the weighted assessment calculation.           | 82.0% for the controlled sample.                   |
| GPA                 | Checks grade-point conversion and credit weighting.   | 10.0 for a single 90% course.                      |
| Invalid score       | Checks range validation.                              | ValidationException is raised.                     |
| Duplicate student   | Checks repository uniqueness.                         | ValidationException is raised.                     |
| Target prediction   | Checks remaining-weight target calculation.           | 80.0% required in the controlled sample.           |
| File persistence    | Checks write/read behaviour for all three data files. | One student, course, and assessment are recovered. |

**Execution command:** java -cp out TestRunner

**Result during project verification:** ALL TESTS PASSED.

## 12. Challenges Faced

Keeping the command-line layer lean while supporting several distinct features meant pushing academic calculations into dedicated service classes — that separation paid off when writing tests.

Weighted assessments needed careful handling because different assessments can have different maximum scores and weights. Putting the score-to-percentage conversion inside the Assessment model kept the logic close to where it belonged.

Persistence needed a stable delimiter and a matching validation rule, since the project doesn't use an external CSV library. Banning the | character from user input was the simplest fix.

During development, two test expectations didn't match the actual calculation outputs. Rerunning the test suite surfaced the mismatches and I corrected them before packaging — a good reminder that tests are worth running before you consider the work done.

## 13. Learnings and Key Takeaways

- Java classes are much easier to maintain when each one has a narrow, well-defined responsibility.
- Business calculations should live away from UI code so they can be tested directly without wiring up a CLI.

- Input validation is application logic, not just a UI concern — it belongs in the service or model layer too.
- Keeping serialization/deserialization logic close to the model or data-store layer makes persistence far simpler to reason about.
- A small project can still have a clear architecture without needing a framework to enforce it.

## 14. Future Enhancements

- Replace flat files with a relational database while keeping the service interface stable.
- Add update and delete operations for students, courses, and assessments.
- Add user authentication and role-based access for students and faculty.
- Export reports to CSV or PDF directly from the application.
- Add sorting, filtering, attendance tracking, and semester-wise analytics.
- Introduce automated JUnit tests if third-party dependencies are allowed by the evaluation environment.

## 15. References

Oracle. Java Platform, Standard Edition documentation. <https://docs.oracle.com/en/java/javase/21/>

Oracle. Java Language Specification, Java SE 21. <https://docs.oracle.com/javase/specs/jls/se21/html/>

Git. Git documentation. <https://git-scm.com/docs>

VITYarthi. Build Your Own Project — General Project Instructions & Submission Guidelines. Provided course PDF.

## Appendix A — Repository Submission Contents

- README.md — setup, execution, testing, data format, and feature documentation.
- statement.md — problem statement, scope, target users, and high-level features.
- src/ — Java source organised into model, repository, service, utility, and exception packages.
- tests/TestRunner.java — repeatable command-line validation.
- data/ — sample records to demonstrate the application immediately after launch.
- docs/ — diagrams, sample terminal output, and submission checklist.
- CampusGrade\_Project\_Report.pdf — the detailed project report.